

Algorithmic Analysis of Water Distribution Networks

Analysis and Design of Advanced Algorithms (Gpo 605)
TC2038, Situation Problem 2

Santiago Hernández Senosiain
A01787550

November 2, 2026

Contents

1 Problem definition **1**

2 Datasets **1**

3 Algorithm **2**

4 Implementation **4**

5 Complexity analysis **5**

6 Results **7**

7 Conclusions **13**

References **15**

1 Problem definition

A water distribution network is an undirected graph $G = (V, E)$ whose vertices are junctions with coordinates $p_v \in \mathbb{R}^2$, a subset $S \subseteq V$ of which are sources, and whose edges are pipes with a capacity $c(u, v) > 0$. The length of a pipe is $\ell(u, v) = \|p_u - p_v\|_2$ and $d_G(u, v)$ denotes the shortest-path distance under ℓ. The Water Graph Problem (Narváez Terán, 2026) poses seven questions on this model, and the situation problem brief (Tecnológico de Monterrey, 2026) asks each student to investigate one of them. With the instructor's approval, this report covers all seven as a single study, because their results feed each other: lengths define distances, distances define sectors, sectors define the flow problems.

T1 Representation. Build adjacency-list and adjacency-matrix representations, explore G with BFS and DFS, find its connected components, and compare time and memory of both representations.

T2 Pipe lengths. Compute $\ell(u, v)$ for every pipe, the shortest, longest and total length, and compare straight-line distance $\|p_u - p_v\|$ with network distance $d_G(u, v)$.

T3 Sectorization. Assign every node to $\sigma(v) = \arg\min_{s \in S} d_G(s, v)$ and find the pipes to close: $C = \{(u, v) \in E : \sigma(u) \neq \sigma(v)\}$.

T4 Freshness. Water age is proportional to d_G. For each sector $V_s = \sigma^{-1}(s)$ find the farthest node $t_s = \arg\max_{v \in V_s} d_G(s, v)$, its distance and its route.

T5 Maximum flow. In each sector, maximise the flow from s to t_s subject to $|f(u, v)| \leq c(u, v)$ and flow conservation, and identify the bottleneck pipes.

T6 Sampling route. Find a minimum-length closed walk that starts and ends at the node labelled office and visits every node. Nodes and pipes may be repeated, since G need not contain a Hamiltonian cycle.

T7 Expansion. Link a new point q by one pipe to its nearest non-source node, $\arg\min_{v \in V \setminus S} \|p_v - q\|$, and update both representations.

2 Datasets

The four instances provided with the challenge (NYT, HAN, FOS, PES) are text files with a `V E` header and four sections: `[NODES]` (`id x y source_flag`), `[EDGES]` (`u v capacity`), `[OFFICE]` (office id, T6) and `[NEW]` (`x y capacity` of nodes to insert, T7). Node attributes are id, coordinates and source flag; the only edge attribute is the third

column, which the slides call the pipe capacity and label as the diameter in their figures (Narváez Terán, 2026). It is used as capacity in T5 and never as length. Table 1 summarises the instances.

Table 1: Instances after validation. Loops: independent cycles $E - V + 1$; Caps.: distinct capacity values.

Inst.	$\|V\|$	$\|E\|$	Src.	Non-src.	Comp.	Loops	Leaves	Max deg.	Caps.	Office
NYT	20	21	2	18	1	2	2	4	9	5
HAN	32	34	2	30	1	3	3	4	2	29
FOS	37	58	2	35	1	22	1	4	22	4
PES	71	98	2	69	1	28	5	5	14	51

Validation. The loader logs, instead of failing on, malformed or non-finite values, pipes to unknown nodes, self-loops, non-positive capacities, repeated pipes, coincident coordinates, id gaps, header mismatches, a missing office, and `[NEW]` points outside the network's bounding box enlarged by 50%. No value is missing or invalid, but three things are unexpected. PES lists pipe 28–30 twice (as `28 30` and `30 28`, same capacity); it is kept once, so PES has 98 pipes, not the declared 99, and that pipe's capacity is not doubled in T5. PES ids have 18 gaps, handled by an id-to-index map. NYT's second `[NEW]` point $(6478, 3050)$ lies about five network widths east of NYT and equals HAN's second entry, so it is treated as a copy error and not inserted.

Structure. All instances are connected and have two sources. NYT and HAN are almost trees, FOS and PES are meshed. Six of the eight sources are leaves (both in NYT and HAN, 37 in FOS, 7 in PES), joined to the network by a single pipe. All four networks are very sparse, with density $2E/(V(V-1))$ between 0.039 (PES) and 0.111 (NYT).

3 Algorithm

Each task is matched to a TC2038 technique that fits its structure and compared with an alternative or baseline.

T1: graph traversal. Connectivity is reachability without weights, so BFS (FIFO queue) and DFS (stack) suffice: each visits every vertex reachable from its start once, and restarting from unvisited vertices yields the components. Both run on the adjacency list, which reads a vertex's neighbours directly, and on the adjacency matrix, which scans a full row.

T2: geometry and dynamic programming. Lengths are one Euclidean distance per pipe. All-pairs network distances come from Floyd-Warshall, a dynamic program over allowed intermediate vertices, $D_{ij}^{(k)} = \min(D_{ij}^{(k-1)}, D_{ik}^{(k-1)} + D_{kj}^{(k-1)})$, with a next-hop matrix for route reconstruction. Circuity $d_G(u, v)/\|p_u - p_v\| \geq 1$ measures how indirect the network is. Length queries are compared on a linear scan (baseline), a hash index and a sorted array with binary search.

T3 and T4: greedy shortest paths. Lengths are non-negative, so Dijkstra's greedy rule (settle the closest unsettled vertex, relax its pipes) is exact. Multi-source Dijkstra starts with every source at distance 0, which equals Dijkstra from a virtual super-source joined to all sources by zero-length pipes; each node is settled by the source that reaches it first, i.e. $\sigma(v)$. The alternative runs Dijkstra once per source and takes the argmin. C is necessary and sufficient to separate the sectors, and closing it does not lengthen any node's route to its own source, because every node on a shortest s–v path with $\sigma(v) = s$ also has $\sigma = s$. T4 reuses the same run: t_s is an $O(V)$ scan of the distances in V_s and its route follows the parent pointers. As the before-sectorization reference, the farthest node a source would have to reach in the open network is its eccentricity.

T5: network flow. Each pipe becomes two opposite arcs of capacity c in a residual graph. Ford-Fulkerson augments along residual s–t_s paths until none remains; the nodes still reachable from s define a minimum cut whose saturated pipes are the bottleneck, and its capacity equals the maximum flow. Edmonds-Karp chooses shortest augmenting paths by BFS; Dinic builds a BFS level graph and saturates it with a DFS blocking flow. Shortest path and maximum flow answer different questions: the first minimises the length of one route and ignores capacity, the second ignores length and asks how much all routes together can carry. One route carries at most its narrowest pipe; the flow is limited by the narrowest cut.

T6: dynamic programming, greedy, local search, branch and bound. With the metric closure $D_{uv} = d_G(u, v)$ (Floyd-Warshall), any feasible walk read as its order of first visits is a tour of D that is no longer, and any tour of D expands into a walk of equal length by replacing each leg with its shortest path. The optimal sampling walk is therefore the optimal tour of D from the office, expanded into real pipes. Nearest neighbour (greedy) walks to the closest unvisited node; 2-opt (local search) replaces legs $(a, b), (c, d)$ by $(a, c), (b, d)$ while that shortens the tour. Branch and bound is exact: a depth-first search over partial routes from the office o, seeded with the 2-opt tour, pruning a partial route ending at u with unvisited set R when

$$\mathrm{LB} = \mathrm{cost} + \mathrm{MST}(R) + \min_{r \in R} D_{ur} + \min_{r \in R} D_{ro} \tag{3.1}$$

reaches the incumbent. Any completion is a path through R (at least MST(R)) plus one leg in and one leg out, so the bound never overestimates. Only branch and bound guarantees optimality.

T7: divide and conquer for geometric search. Brute force compares q with all n non-source candidates. A 2-d tree splits candidates at the median of alternating coordinates; a query descends towards q and visits the far side of a split only if the splitting line is closer than the best distance so far. New points are inserted in file order with the next free id (as node 38 in the slides' FOS example), each linked to the nearest non-source node of the network as it stands, with the record's third value as the pipe's capacity. Updating the list adds one key and two entries; the matrix must be reallocated and copied.

4 Implementation

The code is a Python package, `water`, plus one Jupyter notebook per part (`00_data`, `01` to `07` for the tasks, `08_performance`); each notebook runs top to bottom on its own, locally or in Colab, and writes the figures and tables used here. All algorithms are written by hand; NetworkX and SciPy are used only to cross-check results and Matplotlib only to draw. Claude Code (Anthropic, 2026) assisted with the writing and the code. Module `io` parses and validates the files into a `Network` (coordinates, sources, pipes, office, new nodes); `graph` builds the adjacency list (id → list of (neighbour, length)) and the NumPy adjacency matrix with an id-to-index map, and runs BFS, DFS and components on both; `paths` has Dijkstra with a binary heap (single, repeated, multi-source) and vectorised Floyd-Warshall; `flow` has the residual graph, Edmonds-Karp, Dinic and the minimum cut; `tsp` has nearest neighbour, 2-opt, branch and bound, brute force and walk expansion; `spatial` has brute-force search and the 2-d tree. Module `tasks` holds the chain `sectorize` → `farthest` → `sector_flows`, the metric closure and the expansion; every notebook calls it, so T4 reads its farthest nodes from T3's distances and parents and T5 builds each sector's flow network from T3's open pipes with T4's t_s as sink.

Verification. Every notebook checks its results (Table 2); all checks pass on all instances. Pipe lengths also match, to 0.01, the values printed on the course's figures of FOS, NYT and HAN (Narváez Terán, 2026).

Table 2: Correctness checks, run on all four instances.

T1	List and matrix traversals visit the same set; BFS depths and components equal NetworkX, also after removing a hub's pipes.
T2	Floyd-Warshall equals NetworkX all-pairs Dijkstra; every pipe is the shortest path between its endpoints.
T3	Repeated and multi-source Dijkstra give identical sectors and distances, equal to NetworkX; no exact ties; after closing C the components are exactly the sectors and distances are unchanged.
T4	Each route runs from s to t_s over open pipes with length $d_G(s, t_s)$; NetworkX on each closed sector gives the same maximum.
T5	Edmonds-Karp, Dinic and NetworkX agree; capacity and conservation hold everywhere; min-cut capacity equals the flow.
T6	Branch and bound equals brute force for $n \leq 9$; every walk is closed at the office, covers all nodes, uses real pipes and has the tour's length.
T7	2-d tree equals brute force and SciPy `cKDTree` on 2,000 random queries per instance; updated list and matrix equal rebuilt ones.

5 Complexity analysis

The size parameters are V, E, the number of sources $|S|$ (2 in every instance), the number of locations to visit n (T6, $n = V$ for the full problem) and the number of candidate connections n (T7). Table 3 gives the theoretical cost of each implementation.

Table 3: Theoretical complexity of the implementations (k: pipes reported by a range query).

Task	Algorithm	Time; space
T1	BFS, DFS, components: list; matrix	$O(V + E)$; $O(V^2)$ for both
T2	all pipe lengths	$O(E)$; $O(E)$
	Floyd-Warshall	$O(V^3)$; $O(V^2)$
	length query: scan; hash; sorted array	$O(E)$; $O(1)$; $O(\log E + k)$
T3	repeated Dijkstra (binary heap)	$O(\|S\|(V + E) \log V)$; $O(V + E)$
	multi-source Dijkstra	$O((V + E) \log V)$; $O(V + E)$
T4	farthest node and route, given T3	$O(V)$; $O(V)$
T5	Edmonds-Karp; Dinic	$O(VE^2)$; $O(V^2E)$; space $O(V + E)$
T6	metric closure (Floyd-Warshall)	$O(V^3)$; $O(V^2)$
	nearest neighbour; one 2-opt pass	$O(n^2)$; $O(n^2)$
	branch and bound, MST bound per node	$O(n!\, n^2)$ worst case; $O(n^2)$
T7	brute-force query	$O(n)$
	2-d tree build; query; insert	$O(n \log^2 n)$; $O(\log n)$ expected; $O(\text{depth})$
	graph update: list; matrix	$O(1)$; $O(V^2)$

Measurements. Times are medians of 21 samples (3 for branch and bound), each sample repeating the call for at least 2 ms, on one machine (Python 3.13, Apple arm64). Four instances with $20 \le V \le 71$ are too few to show growth, so growth was measured on induced PES sub-networks grown breadth-first from both sources ($V = 10$ to 71), on office-nearest subsets for T6 ($k = 4$ to 16) and on PES candidate subsets for T7 ($n = 5$ to 69). Figure 1 compares each curve with its cost model scaled by one fitted constant; Table 4 gives the times on the four instances.

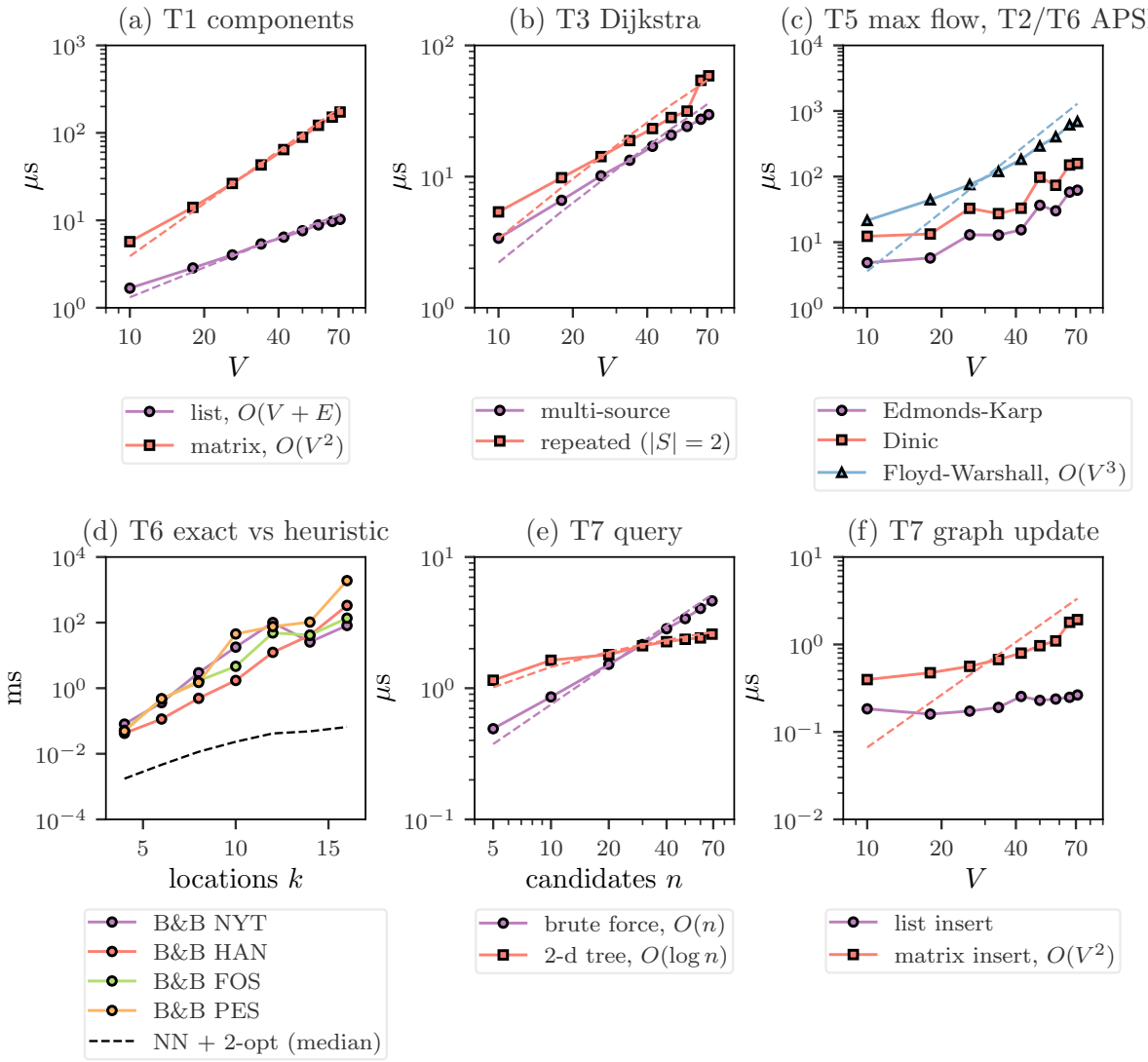


Figure 1: Measured times (solid) against the theoretical cost model scaled to the data (dashed). Log-log axes except (d), whose time axis is logarithmic and whose k axis is linear, so exponential growth appears as a straight line.

Table 4: Median times in microseconds on the four instances. Comp.: all components on list (L) or matrix (M); EK and Dinic: both sectors' flows; FW: Floyd-Warshall; NN+2opt: full route heuristic; NN brute/tree: one nearest-node query.

Inst.	V	E	Comp. L	Comp. M	Dij. multi	Dij. rep.	EK	Dinic	FW	NN+2opt	NN brute	NN tree
NYT	20	21	2.8	17.0	7.2	15.2	12.5	20.0	49.5	101.5	1.4	1.2
HAN	32	34	4.3	37.9	11.0	23.9	17.0	31.6	105.6	271.7	2.2	1.3
FOS	37	58	5.8	52.0	17.4	34.1	47.2	101.0	139.0	530.4	2.6	1.3
PES	71	98	10.3	177.2	29.3	58.4	60.5	115.5	701.1	2748.1	4.7	2.2

Agreement with theory. Most measurements follow their models. Component search grows with log-log slope about 0.9 on the list and 1.8 on the matrix (models 1 and 2), and the matrix is about 17 times slower at $V = 71$. Both Dijkstra variants follow $(V+E)\log V$ (slopes about 1.1 and 1.2); repeated Dijkstra does exactly twice the relaxations of the multi-source version and takes 1.4 times as long on the median subset and 2.0 times on full PES, where fixed overheads matter least. Three measurements deviate, for identifiable reasons. Floyd-Warshall grows with slope about 1.8 instead of 3, because each of its V steps is one vectorised NumPy update whose fixed cost dominates while $V^2 \leq 5{,}041$; it performs exactly V^3 elementary updates and its curve steepens towards the V^3 reference. Edmonds-Karp and Dinic grow almost linearly, far below $O(VE^2)$ and $O(V^2E)$, because every source–sink pair needs at most four augmenting paths, each an $O(E)$ BFS; at this scale Edmonds-Karp is 2.5 times faster, since Dinic's level graph and recursive blocking-flow search only pay off with many augmentations per phase. The matrix insertion grows about 5 times from $V = 10$ to 71, less than V^2, because copying a few thousand cells is cheap next to NumPy's allocation cost. Branch and bound is the only exponential cost: its time multiplies by about 1.4 (NYT) to 2.3 (HAN) per added location, reaching about 2 s at $k = 16$ on PES, while NN + 2-opt stays below 0.1 ms. The 2-d tree query grows logarithmically (3.5 to 11 distance evaluations) and beats brute force from about 30 candidates, but its build only pays for itself after a few tens of queries, so for two or three insertions per instance brute force is sufficient.

6 Results

T1: representation. Every instance is one connected component, so every junction can be supplied by some source. BFS on the matrix is about 8 (NYT) to more than 20 (PES) times slower than on the list, because each visited vertex scans a full row. Memory is close at this size: a list entry is a Python tuple plus a float (about 120 bytes per pipe end) against 8 bytes per matrix cell, so the list is slightly larger on NYT (5.7 against 4.7 KiB), but the matrix grows quadratically and doubles the list on PES (44.6 against 23.3 KiB). The matrix only wins the pipe-existence query (67 against about 135 ns), so the

list is the working representation and the matrix is used only for all-pairs tables.

T2: pipe lengths. Table 5 gives the length statistics. Length and capacity are distinct attributes (correlation -0.65 to $+0.13$). The FOS mesh keeps every pair within 1.75 times its straight-line distance, while NYT and HAN reach 4.73 and 3.32. PES is extreme: nodes 19 and 39 are 147 units apart but 4,146 by pipe, because PES is two dense clusters joined by few pipes. On average network distances are 19% (FOS) to 83% (PES) longer than straight lines, which is why T3 and T4 use d_G. The hash index answers a length query in under 0.1 µs against about 7 µs for a scan, and the sorted array finds the 9 PES pipes longer than 1,000 with 7 comparisons instead of 98.

Table 5: Pipe lengths (coordinate units) and circuity $d_G/\|p_u - p_v\|$ over all node pairs.

Inst.	$\|E\|$	Total	Shortest	ℓ	Longest	ℓ	Mean	Circ. mean	Circ. max
NYT	21	6,020	9-10	182.2	9-16	480.8	286.7	1.44	4.73
HAN	34	33,827	21-22	399.5	29-30	1780.9	994.9	1.32	3.32
FOS	58	56,154	33-34	240.6	5-6	2284.8	968.2	1.19	1.75
PES	98	42,463	26-17	3.6	9-6	1605.9	433.3	1.83	28.20

T3: sectors. Figure 2 shows the sectors (sizes in Table 6). The pipes to close are 2–3, 11–9 and 20–16 in NYT; 7–8 and 14–15 in HAN; 18–19 and 19–20 in PES; and eight in the FOS mesh: 3–4, 28–9, 11–19, 27–16, 11–26, 16–29, 26–27 and 28–29. HAN's sectors are the most unequal (7 against 25 nodes) because source 13 sits behind a single 1,683-unit pipe at the network's edge. Multi-source Dijkstra performed exactly half the relaxations of the repeated version (196 against 392 on PES) and about half the heap pops.

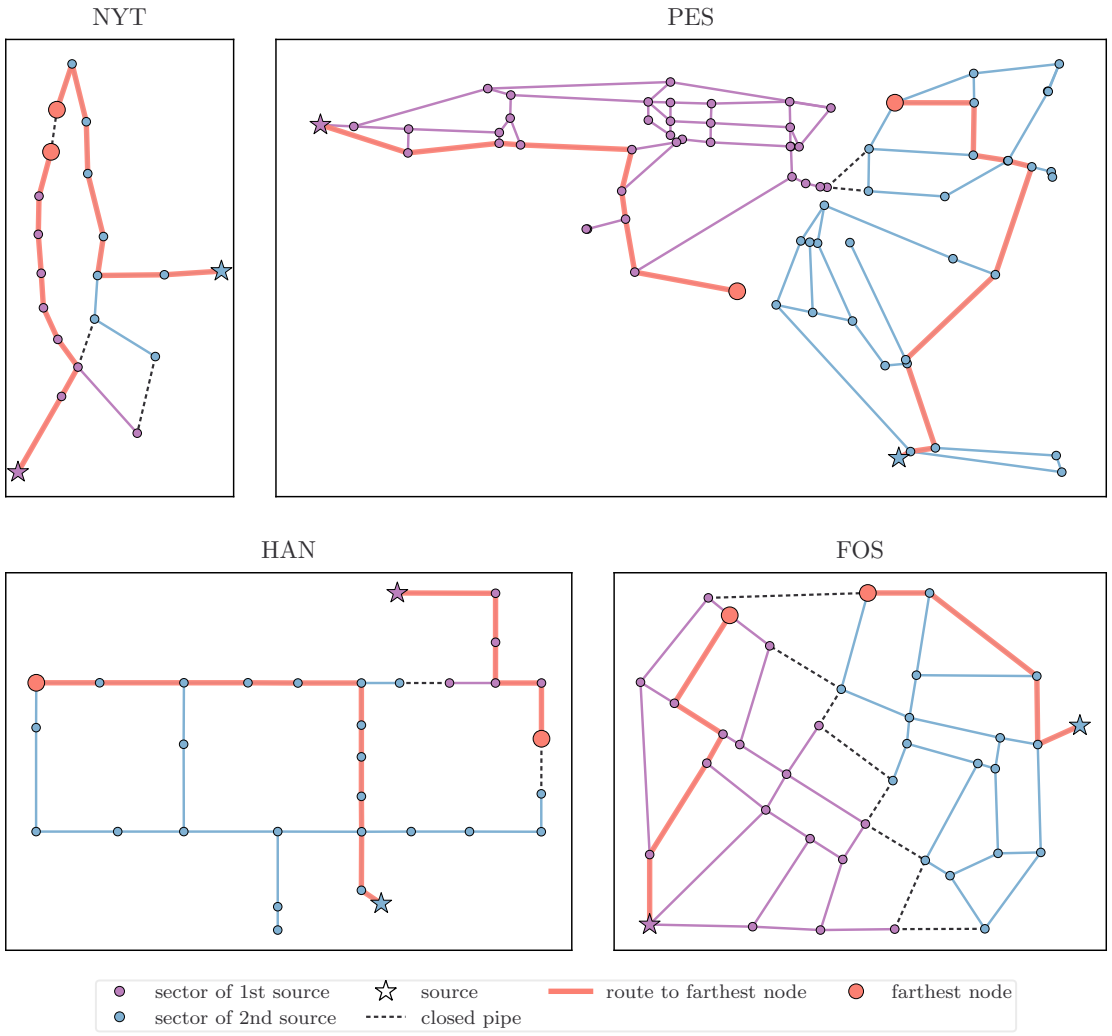


Figure 2: T3 and T4. Sectors from multi-source Dijkstra (colour), closed pipes (dashed) and the shortest route from each source to the node of its sector that receives the oldest water.

T4: freshness. Table 6 gives each sector's farthest node and distance; routes are drawn in Figure 2 and listed in `04_freshness`. Four of the eight (NYT 3 and 2, HAN 8, FOS 3) sit at a closed pipe, where the two supply areas meet; the others lie at the far end of their sector. Column "Before" divides the source's farthest distance in the open network by $d_G(s, t_s)$: sectorization reduces the worst water age a source is responsible for by 1.49 to 2.25 times in FOS and PES and by 2.75 times for HAN source 13; HAN source 1 is unaffected. Per node nothing changes, since closing C leaves every distance to the node's own source intact.

Table 6: Results chained per sector: size (T3), farthest node and distance (T4), maximum flow, capacity of the T4 route alone and minimum cut (T5).

Inst.	s	$\lvert V_s \rvert$	t_s	$d_G(s, t_s)$	Before	Max flow	Route cap.	Min cut: pipe (capacity)
NYT	17	10	3	1,883.3	1.24	73	73	10-17 (73)
NYT	19	10	2	2,101.2	1.11	61	61	18-19 (61)
HAN	13	7	8	4,963.1	2.75	16	16	12-13 (16)
HAN	1	25	31	9,531.2	1.00	16	16	1-2 (16)
FOS	7	19	12	4,931.1	1.49	55	19	5-6 (19), 13-14 (19), 19-20 (17)
FOS	37	18	3	4,131.7	1.82	58.8	17	2-3 (17), 3-11 (41.8)
PES	1	38	10	4,404.1	2.25	151	101	10-11 (151)
PES	7	33	21	4,529.3	2.01	101	101	6-7 (101)

T5: maximum flow. In six of the eight sectors the minimum cut is the single pipe at a leaf source (NYT, HAN, PES source 7) or leaf sink (PES 10), so the rest of the sector does not affect throughput. Comparing with the T4 route shows the difference between the two problems: PES source 1's shortest route carries only 101, but a second route into node 11 lets 151 reach node 10, and in the FOS mesh three disjoint bottleneck pipes carry 55 against 19 on the shortest route, and 58.8 against 17 in the other sector (Figure 3). Making the cut pipes unlimited shows how capacity constrains the flow: the gain is nil in HAN, where every route is a chain of 16-capacity pipes, and reaches 3.6 times for FOS source 37 (58.8 to 211). Edmonds-Karp and Dinic always agree, with at most 4 augmenting paths.

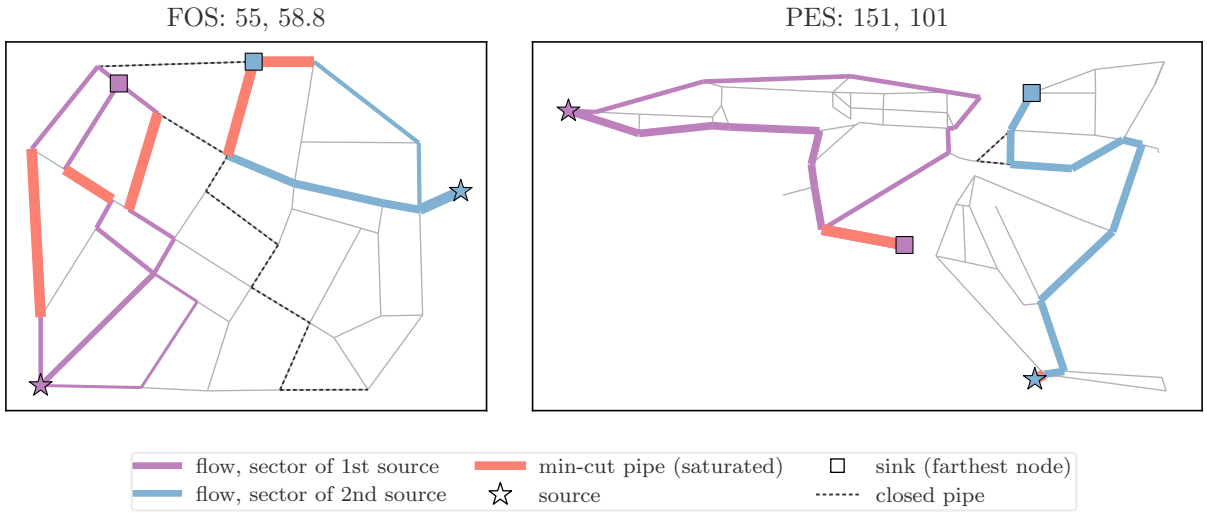


Figure 3: T5 on the two meshed instances. Pipe width is proportional to flow; saturated minimum-cut pipes are highlighted. Titles give each sector's maximum flow.

T6: sampling routes. Table 7 compares the methods. NYT is solved exactly: the optimal walk has length 7,072 over 24 pipe traversals and walks both source branches there and back (Figure 4a). For HAN, FOS and PES branch and bound does not finish within 60 s, so their routes are 2-opt local optima, approximate and not certified: the MST bound is weak because it counts each connection once, while the walk must come back out of every dead end (HAN: 43 traversals of 34 pipes). On office-nearest subsets of $k = 4, 6, \ldots$ nodes, solved exactly up to $k = 18$ to 24, nearest neighbour is on average 15.9% above the optimum (worst 37.8%) and 2-opt 1.8% (worst 15.8%), optimal in 31 of 39 subsets (Figure 4c).

Table 7: Sampling routes. 2-opt gap: relative to the optimum when proven, otherwise an upper bound relative to the root lower bound. Walk pipes: pipe traversals of the expanded route.

Inst.	n	NN	2-opt	B&B	Status	Lower bound	2-opt gap	Walk pipes
NYT	20	9,581	7,622	7,072	optimal	5,535	7.8%	24
HAN	32	43,802	40,810	40,810	best found	30,605	33.3%	43
FOS	37	40,130	30,424	30,424	best found	25,147	21.0%	41
PES	71	38,604	35,510	35,510	best found	22,195	60.0%	97

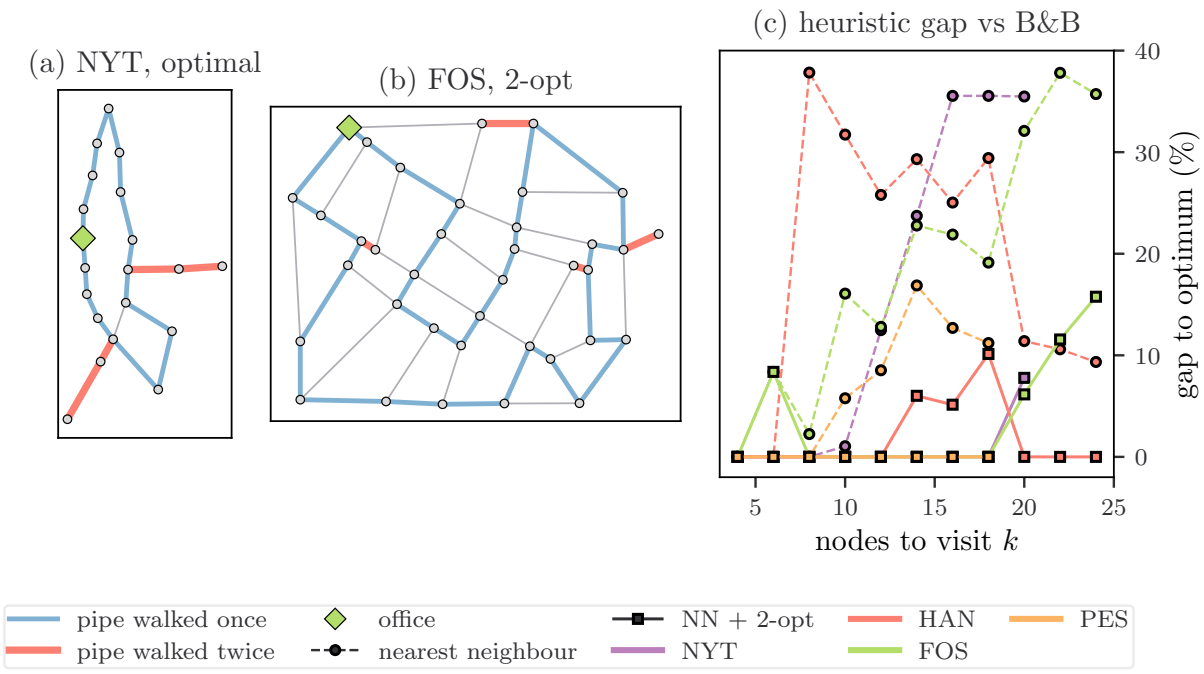


Figure 4: T6. (a) Optimal NYT walk (7,072) and (b) best FOS walk found (30,424) from the office; pipes walked twice are highlighted. (c) Gap of nearest neighbour (dashed) and NN + 2-opt (solid) to the branch-and-bound optimum on office-nearest subsets.

T6 extension: a selection hyper-heuristic. A hyper-heuristic searches over heuristics rather than solutions: a high-level rule chooses, at each step, which low-level heuristic

to apply, and an acceptance criterion decides whether to keep the result. Here four random tour moves (2-opt segment reversal, swap, relocate, and or-opt block moves of 2 or 3 locations) are selected either uniformly or adaptively, by probability matching on each move's recent success rate with a 5% floor. Candidates are accepted by late acceptance hill climbing (no worse than the current tour or than the tour 50 steps earlier). Each run makes 30,000 moves from the nearest-neighbour tour; each configuration is run with 10 seeds and compared with every move used alone under the same acceptance and budget (`09_hyper_heuristic`). All 840 runs return valid tours, none below a proven optimum. On the 11 problems with a proven optimum (Figure 5b) both hyper-heuristics average about 0.2% above it and are optimal in about 94% of runs, slightly ahead of the best single move (2-opt alone, 0.29%) and with a smaller worst case (4.66% against 7.96%); nearest neighbour plus 2-opt averages 1.73%. A run takes about 0.17 s. Their main gain is consistency on the full instances: random selection reaches the best known HAN and FOS routes in every run, whereas 2-opt alone has medians 2.2% and 5.5% above them. On PES it finds the shortest route of the study, 34,700, 2.3% below the T6 route, although its median is slightly worse than 2-opt alone (Figure 5a). Adaptive selection ends up nearly uniform (about 25% per move) and brings no benefit, because near a local optimum all moves rarely improve and their credit decays to the same floor. The routes remain uncertified for HAN, FOS and PES.

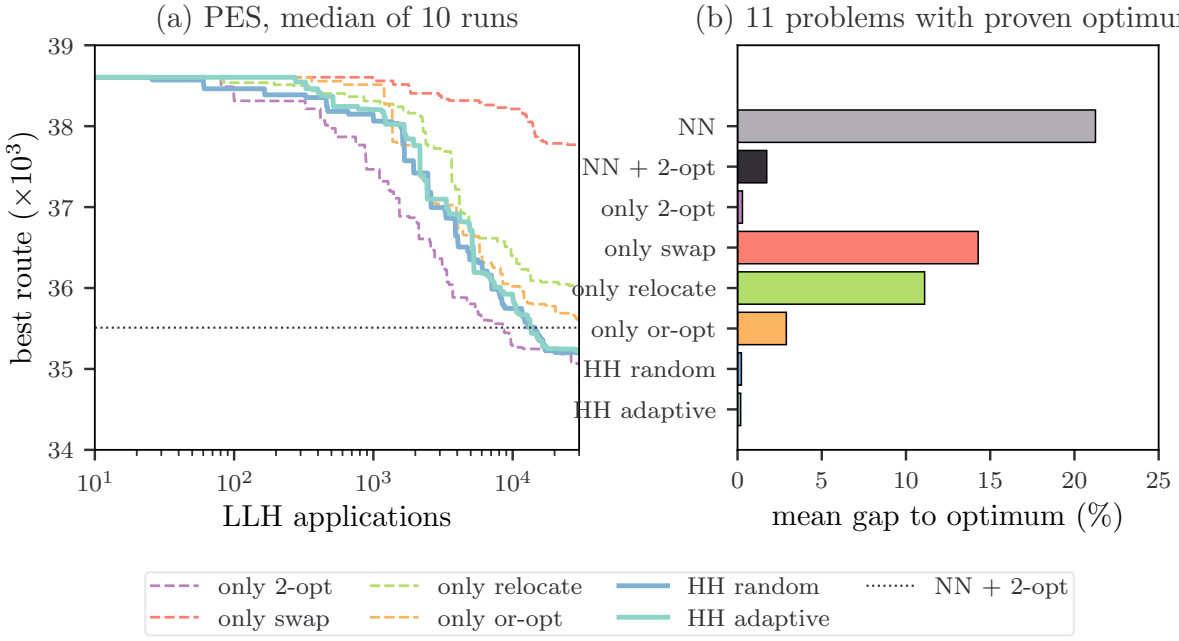


Figure 5: Selection hyper-heuristic for T6. (a) Best PES route over the run (median of 10 seeds) for each move alone (dashed) and for the hyper-heuristics (solid). (b) Mean gap to the proven optimum.

T7: expansion. Table 8 lists the insertions (Figure 6). The FOS point (7071, 9055) becomes node 38 linked to node 17, the connection drawn in the challenge slides. No

new node is closest to an earlier new node, so insertion order does not matter here. On 2,000 random queries the 2-d tree evaluated on average 6.7 of 18 candidates on NYT and 11.0 of 69 on PES. Each insertion writes three list entries, whereas the matrix copies the whole table (10,225 cells for the two PES insertions).

Table 8: New nodes, their nearest non-source node and the distance evaluations of each search.

Inst.	New id	Coordinates	Linked to	Length	Capacity	Tree eval.	Brute eval.
NYT	21	(930, 1779)	13	150.0	120	4	18
NYT	22	(1269, 1069)	20	217.0	170	4	19
HAN	33	(8457, 5177)	9	353.0	18	5	30
HAN	34	(6478, 3050)	5	607.7	20	9	31
HAN	35	(595, 5929)	32	793.5	17	6	32
FOS	38	(7071, 9055)	17	617.1	50	5	35
FOS	39	(1957, 9160)	5	809.1	45	7	36
PES	90	(657029, 963601)	3	363.7	120	9	69
PES	91	(662127, 961448)	25	645.7	105	10	70

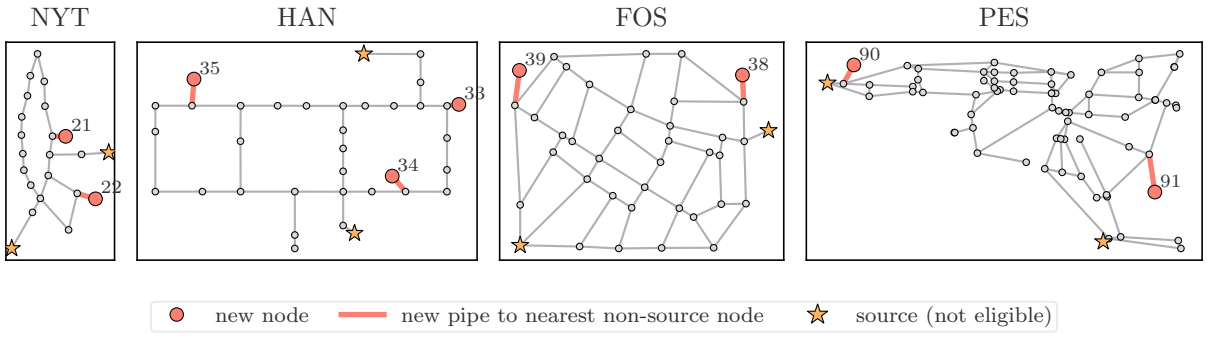


Figure 6: T7. Expanded networks: each new node is linked to its nearest non-source node.

7 Conclusions

How the tasks relate. The tasks form a chain. T2's lengths are the weights of every shortest path; T3 turns them into sectors; T4 reads the oldest-water node of each sector from the same Dijkstra tree; T5 uses that node as the sink of a flow problem restricted to the sector. T6 reuses T2's all-pairs distances as its metric closure, and T1 decides the representation all of them run on. The results also explain each other. The leaf sources found in the data stage are both the single-pipe bottlenecks of T5 and the dead ends the T6 walk must traverse twice. PES's extreme circuity in T2 reflects two clusters joined by few pipes, which is why separating its sectors needs only two closures in T3. T4 and T5 on the same source–sink pairs show concretely that the shortest route and the maximum

flow are different problems: the route minimises length, the flow is set by the narrowest cut, and in the meshed FOS the flow is about three times what the shortest route alone could carry.

Cost and difficulty. T6 is the only hard problem. Its exact solution grows exponentially with the number of locations, so optimality was proven only for NYT and for subsets of at most 24 nodes; for HAN, FOS and PES the routes are heuristic. Among the polynomial steps, Floyd-Warshall ($O(V^3)$) is the most expensive, and the adjacency matrix in general costs $O(V^2)$ where the list costs $O(V + E)$. Dijkstra, the flow algorithms and the nearest-node search are all fast at this scale; the flow algorithms run far below their worst-case bounds because the sectors need at most four augmenting paths. The cheap heuristics carry most of the value of the expensive method: 2-opt was within 1.8% of the optimum on average. A selection hyper-heuristic over the same kind of moves made the heuristic routes more reliable across runs and shortened the best PES route by 2.3%, still without a guarantee.

Limitations. The model is purely graph-theoretic: there is no hydraulics (pressure, head loss, demand), capacities are taken as given numbers without units, and water age is taken as proportional to distance rather than to travel time. Flow is analysed for one source–sink pair per sector inside the closed sector, not for the whole network. Several conventions were needed and are stated where used: the treatment of PES's repeated pipe, the rejection of NYT's outlying new point, the sequential insertion of new nodes, and tie-breaking towards the earlier source (no ties occurred). The branch-and-bound bound is simple and weak on near-tree networks, so the full HAN, FOS and PES routes remain uncertified. Finally, the instances are small ($V \leq 71$), so performance conclusions rest on growth across subsets and are affected by Python interpreter overhead.

References

Anthropic (2026). Claude code. AI coding assistant, `https://www.anthropic.com/claude-code`. Used for assistance with writing and code.

Narváez Terán, V. (2026). Problem situation 2: Distribution of water. Lecture slides, Análisis y diseño de algoritmos avanzados, Tecnológico de Monterrey. Includes the NYT, HAN, FOS and PES network instances.

Tecnológico de Monterrey (2026). TC2038 advanced algorithms design. situation problem 2: Algorithmic analysis of water distribution networks. Course assignment brief.